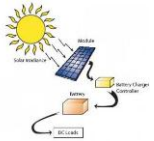


Cortex-M微控制器基础知识

Cortex-M4学习参考资料：

（1）ARM Cortex M3与M4权威指南，（第三版） Joseph Yiu著，吴常玉等译，清华大学出版社；
（2）Cortex M3权威指南，Joseph Yiu 著，宋岩译
（3）嵌入式系统设计基础及应用，郭建等著，清华大学出版社；
（4）精通STM32F4-库函数版，张洋等编著，北航出版社。
（5）STM32F4xx中文参考手册，pdf文件
（6）STM32F405xx STM32F407xx Datasheet.pdf



内容

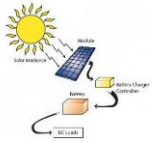
（1）ARM微处理器介绍

（2）ARM 微处理器架构

（3）指令集

（4）异常和中断

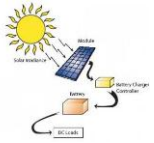
（5）存贮器系统



（1）ARM微处理器介绍

ARM 公司概况

- ARM founded in November 1990
  - **Advanced RISC Machines**
- Company headquarters in Cambridge, UK
  - Processor design centers in Cambridge, Austin, and Sophia Antipolis
  - Sales, support, and engineering offices all over the world
- Best known for its range of RISC processor cores designs
  - Other products – fabric IP, software tools, models, cell libraries
  - to help partners develop and ship ARM-based SoCs
- ARM does *not* manufacture silicon，无晶圆厂（Fabless）模式
- Arm partners have shipped **more than 250 billion Arm-based chips** (2023).



1.1 ARM 与 Cortex

- ARM 处理器在Cortex家族发布前的常用系列
  ARM7，ARM9，ARM11
- 2004年后采用Cortex命名ARM处理器，分类：

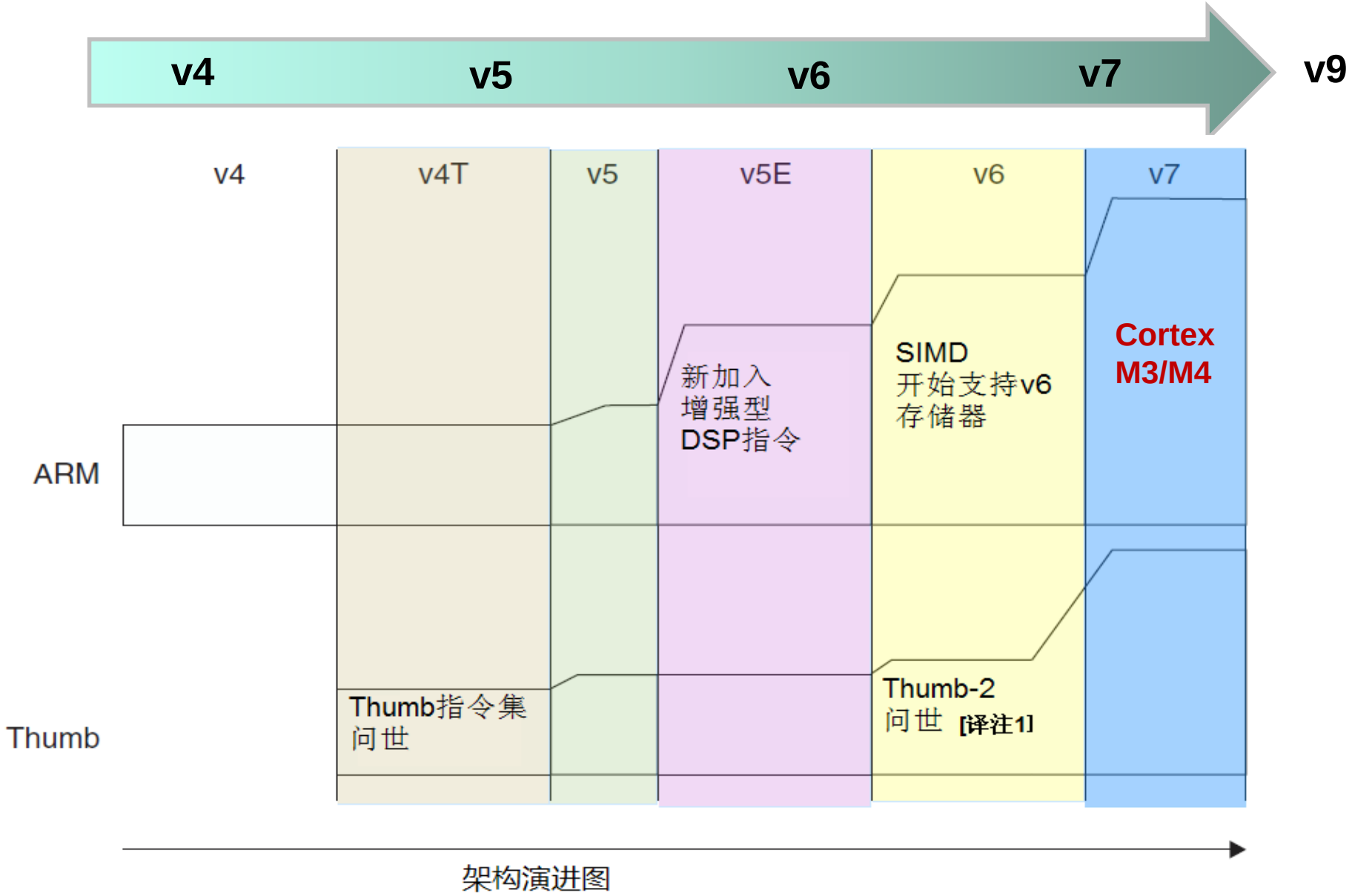
➢ Cortex-A 系列用于高性能开放应用平台
(high-performance open Application platforms)

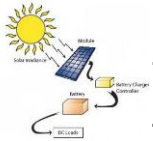
➢ Cortex-R系列用于实时控制
(embedded systems with high Real-time performance)

➢ Cortex-M系列用于传统微控制器
(embedded Microcontroller-type systems)

2020年3季度共出货： 6.7 billion Arm-based chips
其中包括： 4.4 billion Cortex-M processors

1.2 ARM 架构版本


v4
v5
v6
v7
v9
v4
v4T
v5
v5E
v6
v7
ARM
Thumb
新加入
增强型
DSP指令
SIMD
开始支持v6
存储器
Cortex
M3/M4
Thumb指令集
问世
Thumb-2
问世 [译注1]
架构演进图

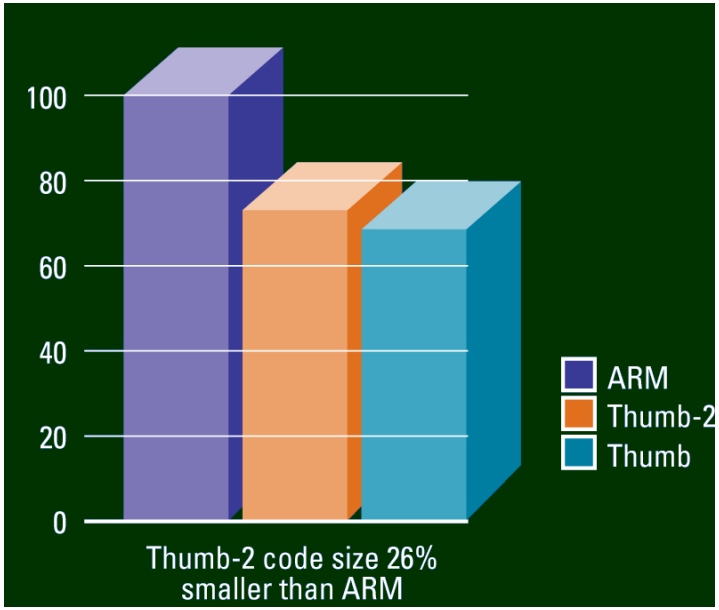


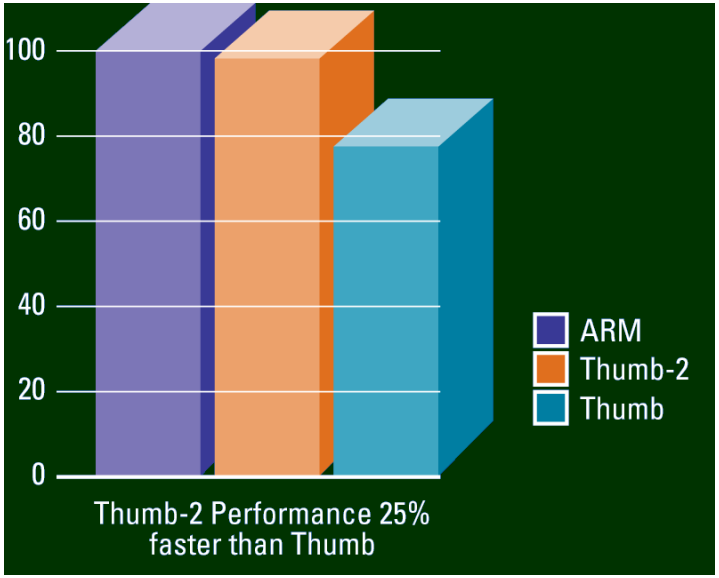
1.3 Cortex-M3/M4处理器数据宽度

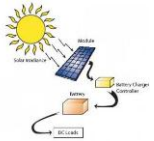
- ARM Cortex-M为32位RISC处理器
  - 32位寄存器
  - 32位内部数据
  - 32位总线接口
- ARM Cortex-M采用32位寻址，地址空间4GB
- 除了32位数据，还可以处理
  - **字节/Byte** (8 bits)
  - **半字/Halfword** (16 bits= 2 bytes)
  - **字/Word** (32 bits =4 bytes)
  - **双字/Doubleword** (64 bits =8 bytes)

1.4 Cortex-M3/M4指令集

- 传统ARM 的指令集包括
  - 32位的ARM指令（功能强，速度快）
  - 16位的Thumb指令（代码小）
  - ARM指令和Thumb指令工作不同模式，需要切换
- Cortex采用支持Thumb2
  - 同时支持ARM指令和Thumb指令
  - 兼顾了指令密度和指令速度（右图）
- 具有3级流水线
  - 取指
  - 译码
  - 执行







1.5 流水线结构

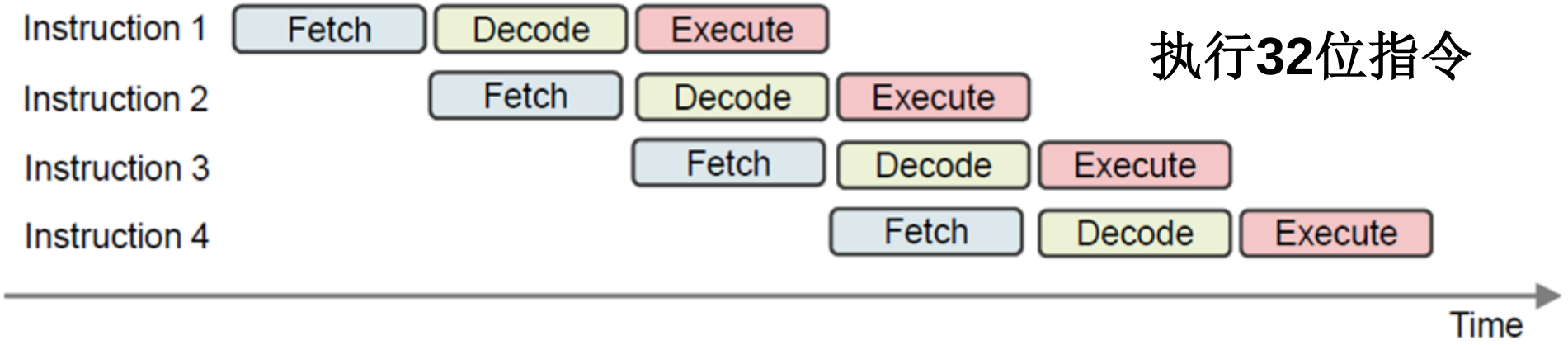
- **核心定义和类比**
  - 如同汽车组装线，将复杂的指令处理过程拆分为多个独立过程，**允许硬件同时处理多条指令的不同部分**。

- **设计目标**
  - 提高吞吐量（单位时间内完成的指令数），而非缩短单条指令的绝对执行时间。

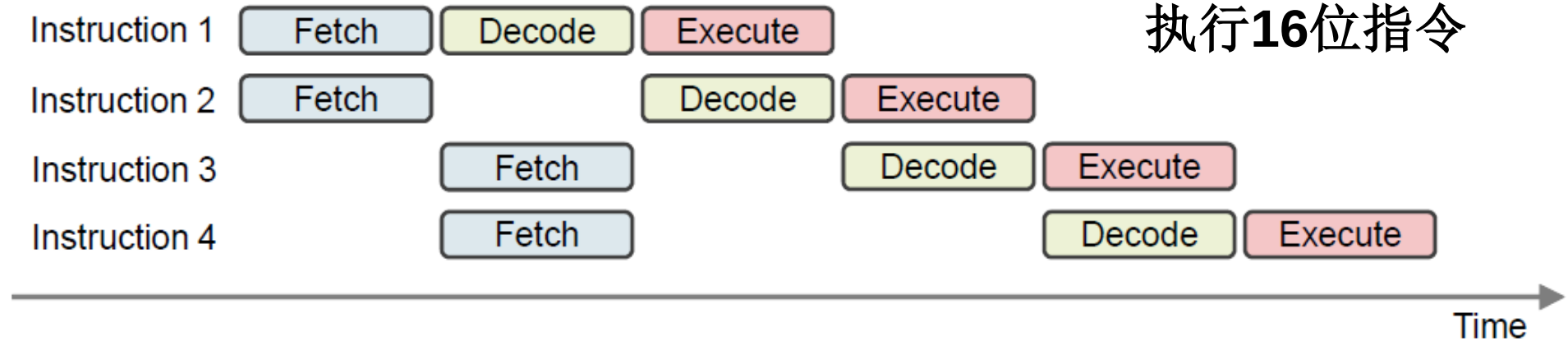
- **经典流水线模型**



1.5 三级流水线



执行32位指令



执行16位指令

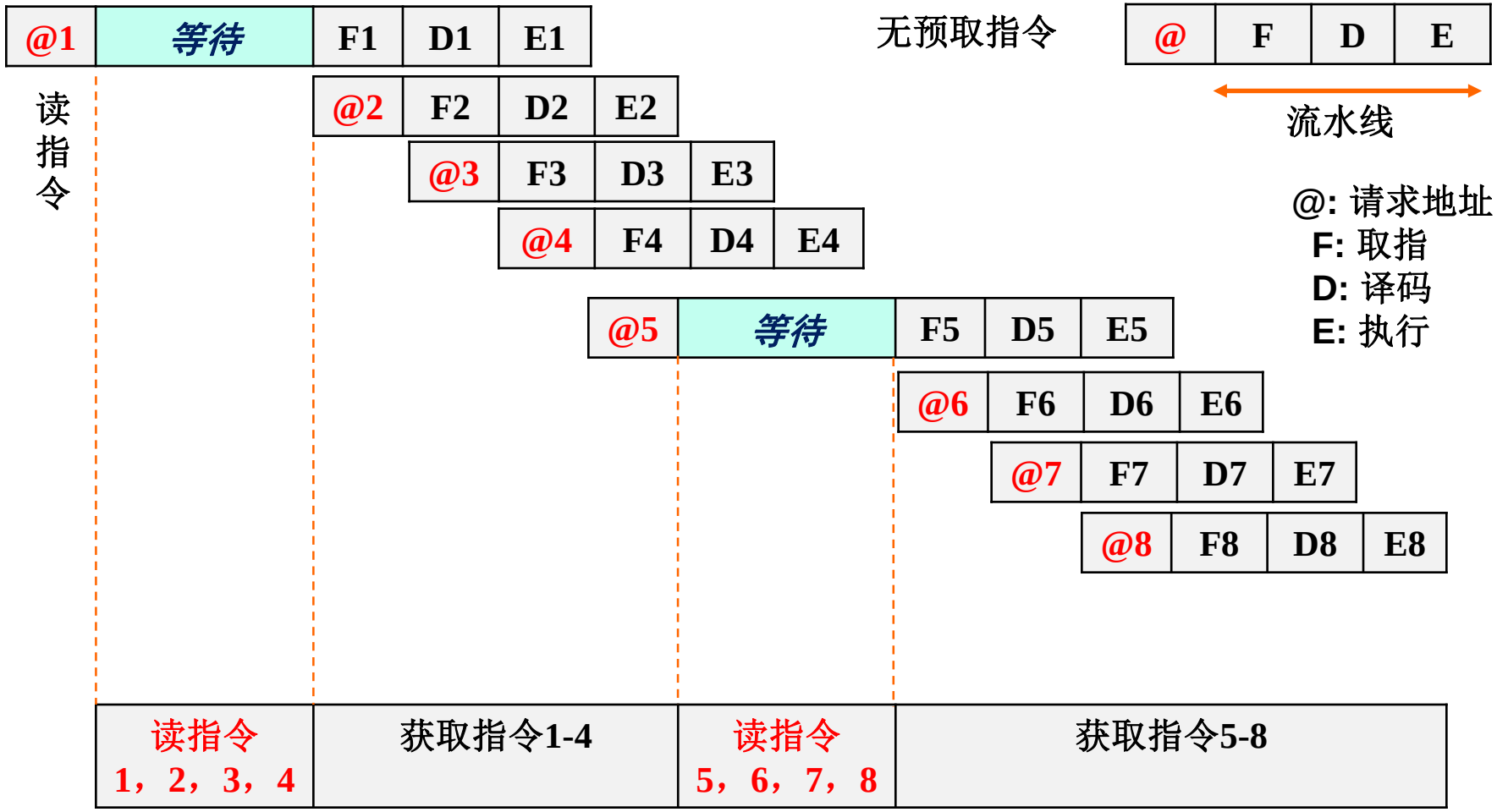
- 某些指令执行多个周期, 此情况下流水线暂停;
- 跳转到分支后流水线清空;
- 可一次读取两条指令(16-bit 指令)

影响指令执行时间的因素

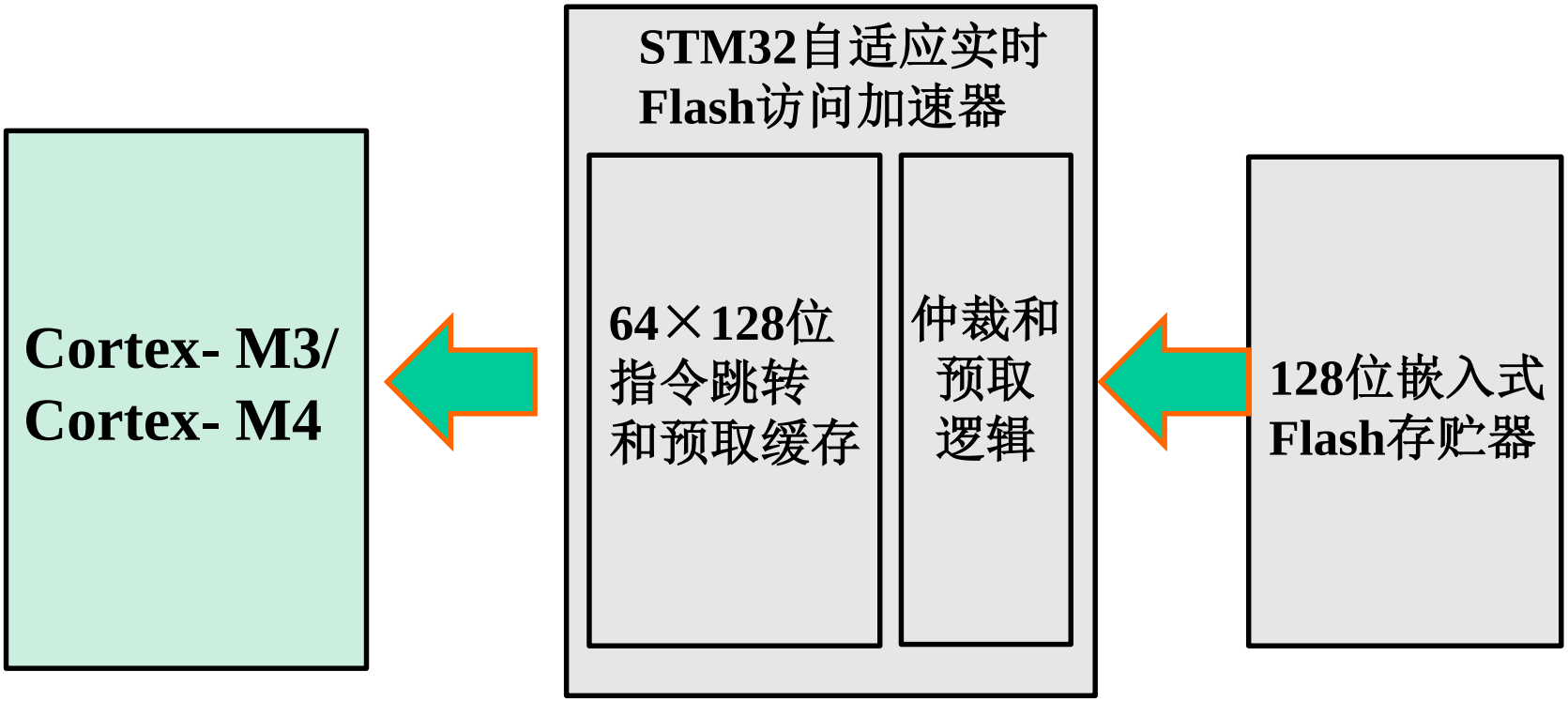
– **Flash访问速度限制**

Flash的读取速度较慢，典型访问时间在20-50 ns范围内

解决策略：增加Flash读取数据宽度（128bit）和指令预取



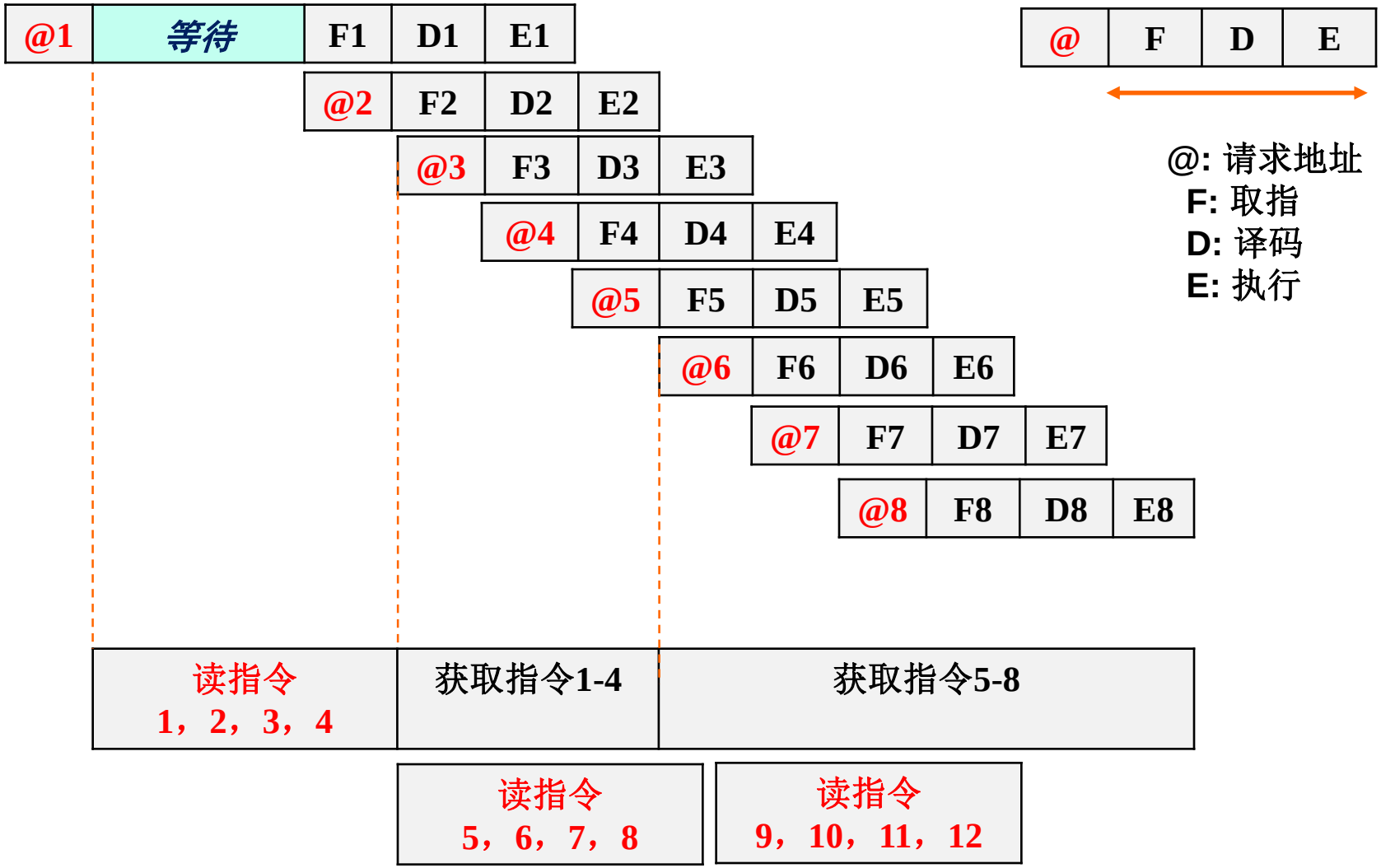
STM32F4的Flash访问加速器



Cortex M3/M4 的Flash访问机制

影响指令执行时间的因素

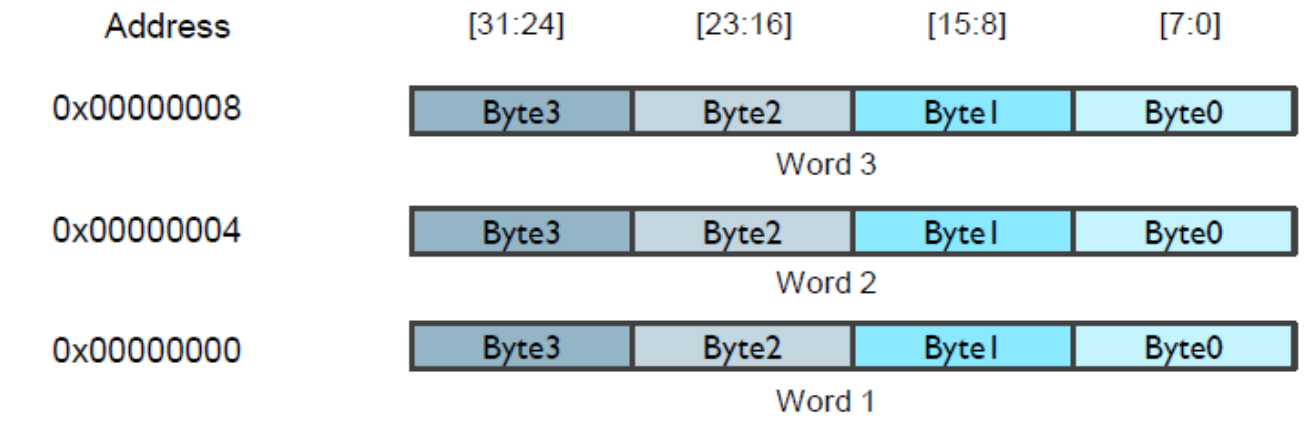
–有指令预取的时序



1.6 Cortex-M3/M4 字节顺序

Endian refers to the order of bytes stored in memory

每个字节对应一个地址



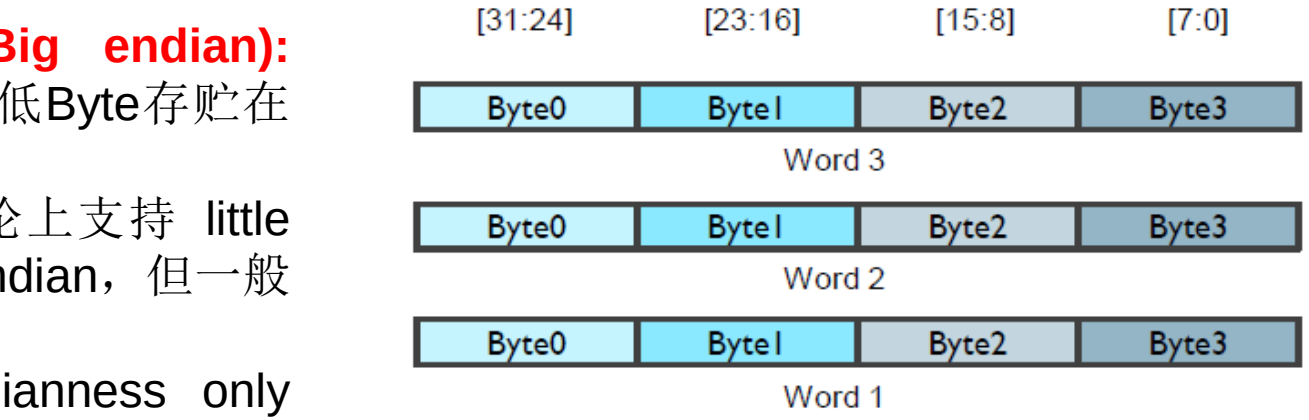
Little endian 32-bit memory

– **小端模式(Little endian)**: 一个Word中的最低Byte存贮在bit 0 to bit 7

– **大端模式(Big endian)**: 一个Word中的最低Byte存贮在bit 24 to bit 31

• Cortex-M4 理论上支持 little endian 和 big endian，但一般采用 little endian

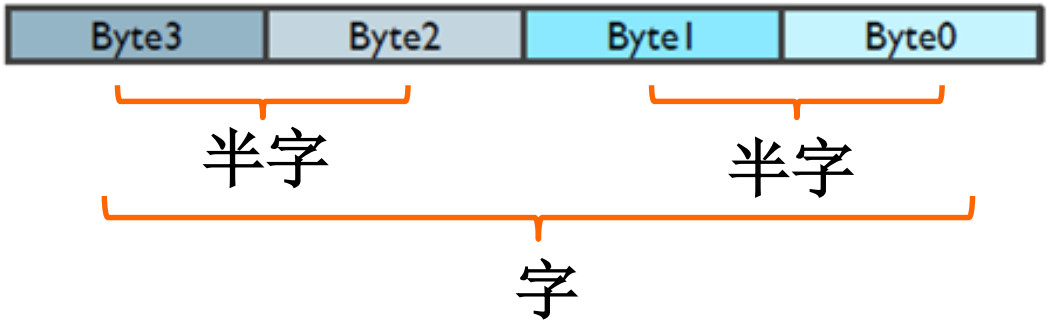
• However, Endianness only exists in the hardware level



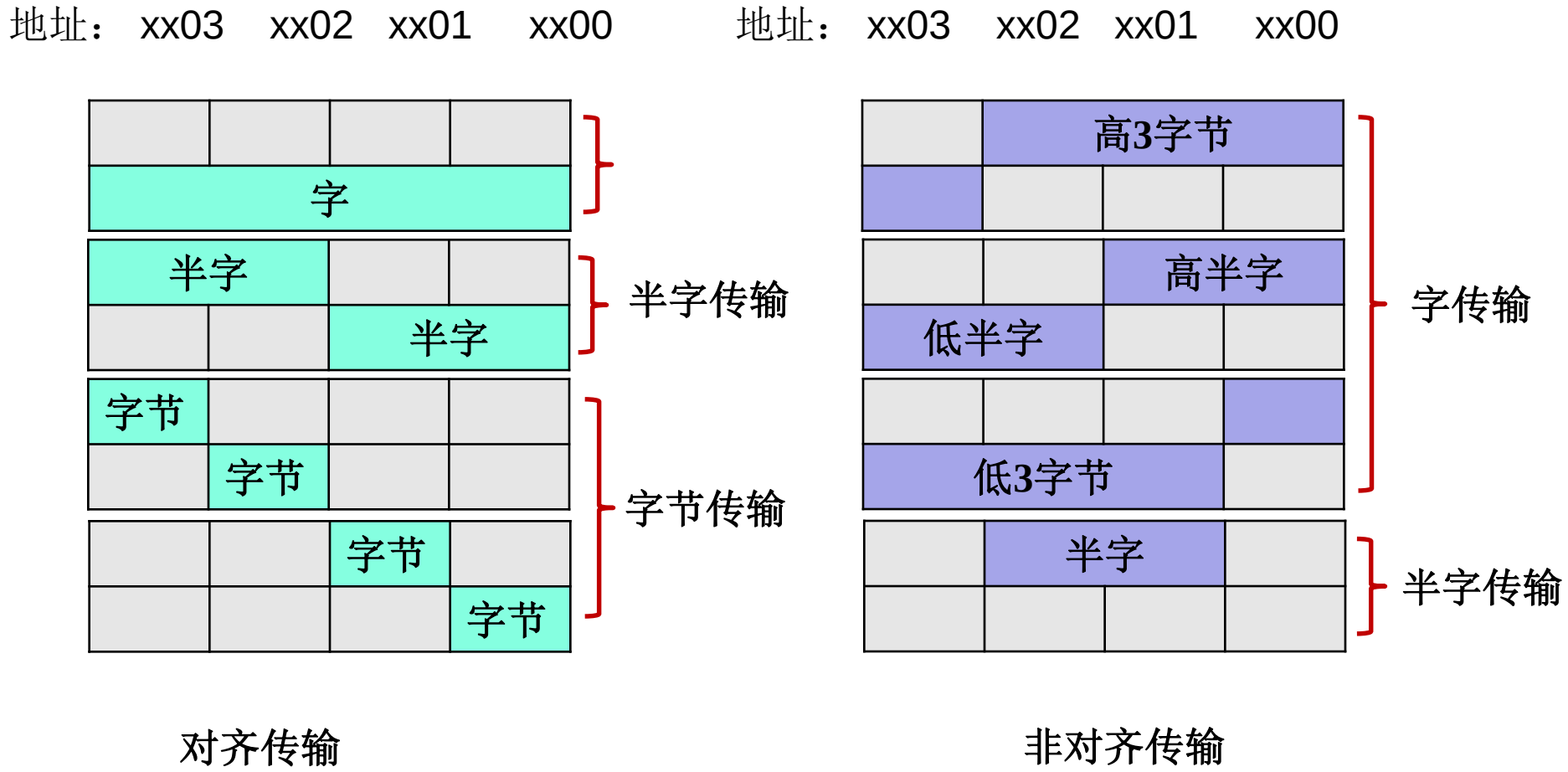
Big endian 32-bit memory

数据地址的对齐

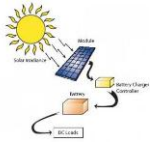
- 存贮器系统32位（4字节），地址**按字节**定义
- 访问方式包括字（32位），半字（16位），字节（8位）
- 对齐传输方式：
  字变量的地址必须是4的整数倍；
  半字变量的地址必须是2的整数倍。
- 非对齐传输方式：
  对字和半字的地址无限制



数据对齐和非对齐访问支持



通常采用对齐方式



1.7 存贮器保护单元（MPU）

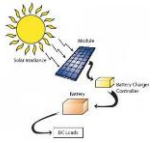
- MPU是一种**硬件机制**，用于增强嵌入式系统的安全性、可靠性和调试能力。它通过定义内存区域的访问权限和属性，防止未经授权的访问或非法操作，从而保护系统资源，特别是在多任务环境（如RTOS）中的运行。

- **MPU作用**：它允许程序定义多个内存区域（region），并为每个区域设置访问权限（如读/写/执行）和内存属性（如缓存行为）。这可以：

➢ 防止任务访问未授权的内存（隔离任务）。

➢ 避免非法操作导致系统崩溃（提升可靠性）。

➢ 辅助调试（如捕获非法访问）。



存贮器保护单元（MPU）

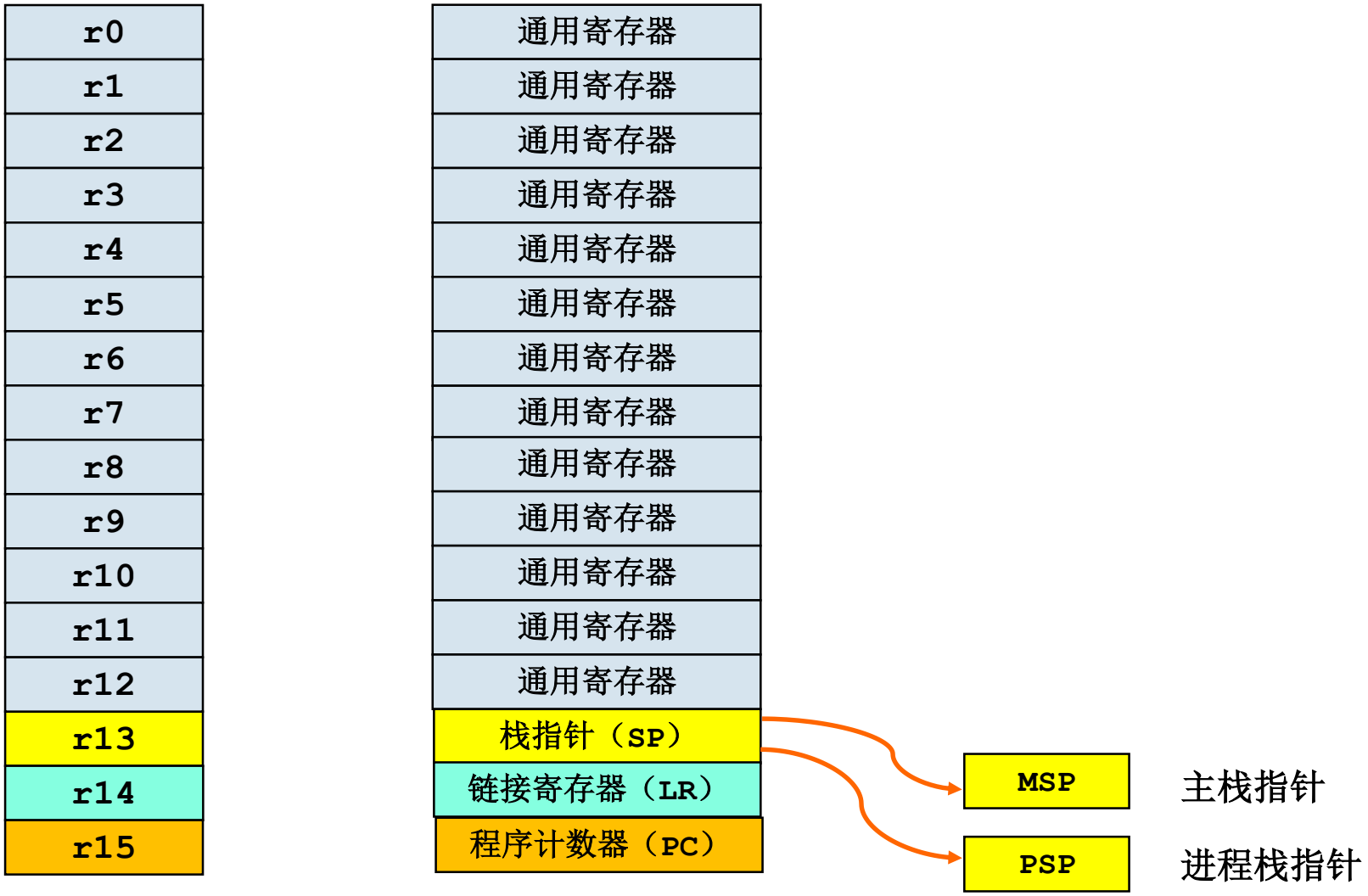
存贮器访问缺省设置（STM32 M3）

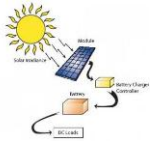
存储器区域	地址范围	用户级许可权限
代码区	0000_0000 – 1FFF_FFFF	无限制
片内 SRAM	2000_0000 – 3FFF_FFFF	无限制
片上外设	4000_0000 – 5FFF_FFFF	无限制
外部 RAM	6000_0000 – 9FFF_FFFF	无限制
外部外设	A000_0000 – DFFF_FFFF	无限制
ITM	E000_0000 – E000_0FFF	可以读。对于写操作，除了用户级下允许时的 stimulus 端口外，全部忽略
DWT	E000_1000 – E000_1FFF	阻止访问，访问会引发一个总线 fault
FPB	E000_2000 – E000_3FFF	阻止访问，访问会引发一个总线 fault
NVIC	E000_E000 – E000_EFFF	阻止访问，访问会引发一个总线 fault。但有个例外：软件触发中断寄存器可以被编程为允许用户级访问。
内部 PPB	E000_F000 – E003_FFFF	阻止访问，访问会引发一个总线 fault
TPIU	E004_0000 – E004_0FFF	阻止访问，访问会引发一个总线 fault
ETM	E004_1000 – E004_1FFF	阻止访问，访问会引发一个总线 fault
外部 PPB	E004_2000 – E004_2FFF	阻止访问，访问会引发一个总线 fault
ROM 表	E00F_F000 – E00F_FFFF	阻止访问，访问会引发一个总线 fault
供应商指定	E010_0000 – FFFF_FFFF	无限制

（2） ARM 处理器架构

2.1 寄存器

通用寄存器组





2.1 寄存器

通用寄存器：

R0-R7：低寄存器（一些16位指令只能访问低寄存器）
R8-R12：高寄存器（可用于32位和16位指令）

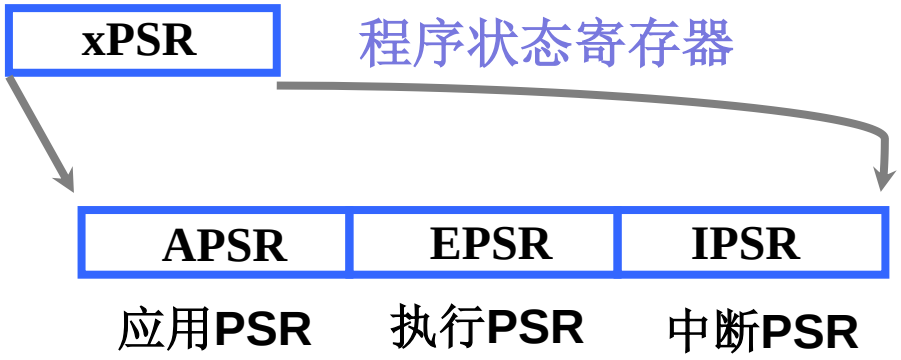
R13： 栈指针（SP）
用于PUSH和POP操作，物理上实际存在两个栈指针：
主栈指针MSP和进程栈指针PSP，不使用操作系统时仅用MSP

R14：链接寄存器（LR）
用于函数或子程序调用时返回地址的保存。当执行了函数或子程序调用时，LR的数值会自动更新。

R15：程序计数器（PC）
可读可写，读操作返回当前指令地址加4；
写PC引起跳转操作。

2.1 寄存器

特殊寄存器



在使用C语言开发应用程序时，需要访问这些寄存器的场景不多。



中断/异常
屏蔽寄存器

特殊寄存器未经存贮器映射，不能直接访问。
但可用MRS、MSR指令访问。

MRS <reg>,<special_reg>

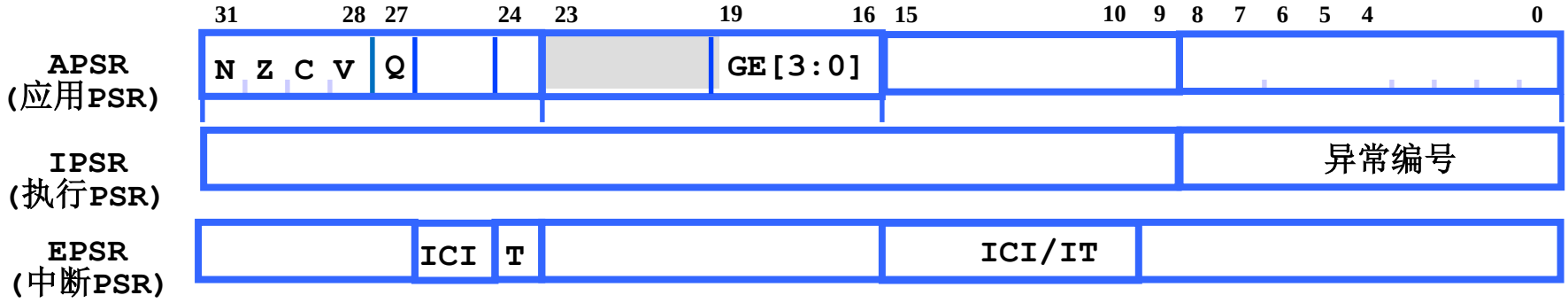
MSR <special_reg>, <reg>

CONTROL

处理器控制

2.1 寄存器

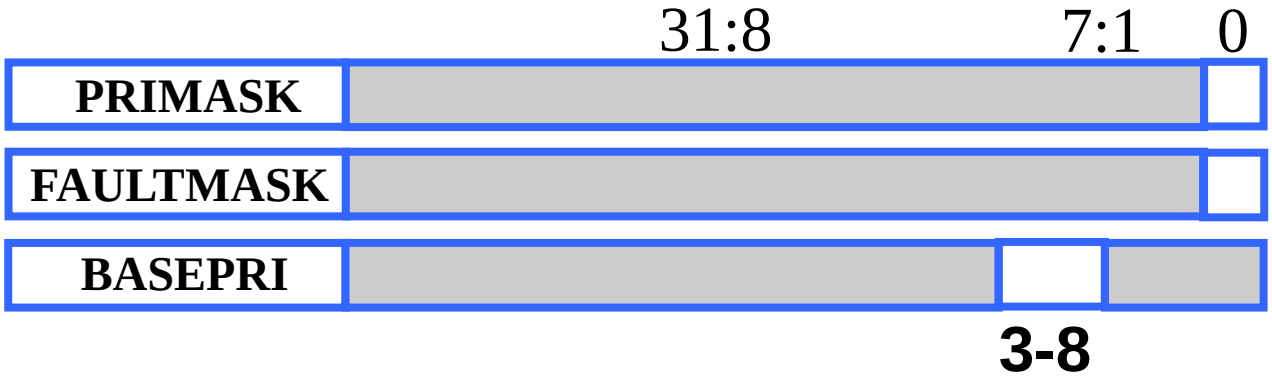
程序状态寄存器 xPSR（类似8051中的PSW）



- 状态标志
  - N = Negative result from ALU (结果为负数，即bit[31]=1)
  - Z = Zero result from ALU (零标志，运算结果为0)
  - C = ALU operation Carried out (无符号数运算溢出标志)
  - V = ALU operation oVerflowed (有符号数运算溢出）
  - Q = Sticky Overflow flag （饱和运算状态标志）
- – GE[3:0] = 针对每个字节的大于或等于标志，仅在M4中有用
- – ICI/IT =中断继续指令，IF-THEN 指令状态位用于条件执行
- – T = Thumb 状态，等于1，清除此位将引起异常错误

2.1 寄存器

PRIMASK、FAULTMASK、BASEPRI寄存器



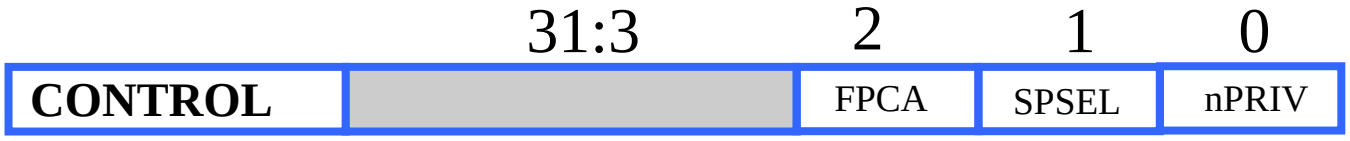
名字	功能描述
PRIMASK	一位有效，置1后关闭所有可屏蔽的异常，仅剩不可屏蔽中断(NMI)和硬件故障（HardFault）可以响应。缺省值为0，表示没有关中断。
FAULTMASK	一位有效，置1后关闭所有可屏蔽的异常和HardFault异常，只有NMI才能响应。缺省值也为0，表示没有关异常。
BASEPRI	3-8位有效（由异常的优先级位数决定）。它定义了被屏蔽优先级的阈值。当它被设成某个值后，所有优先级号大于等于此值的中断都被关闭（优先级号越大，优先级越低）。但若被设成0，则不关闭任何中断，缺省值为0。

只有在特权等级才能对其操作！

2.1 寄存器

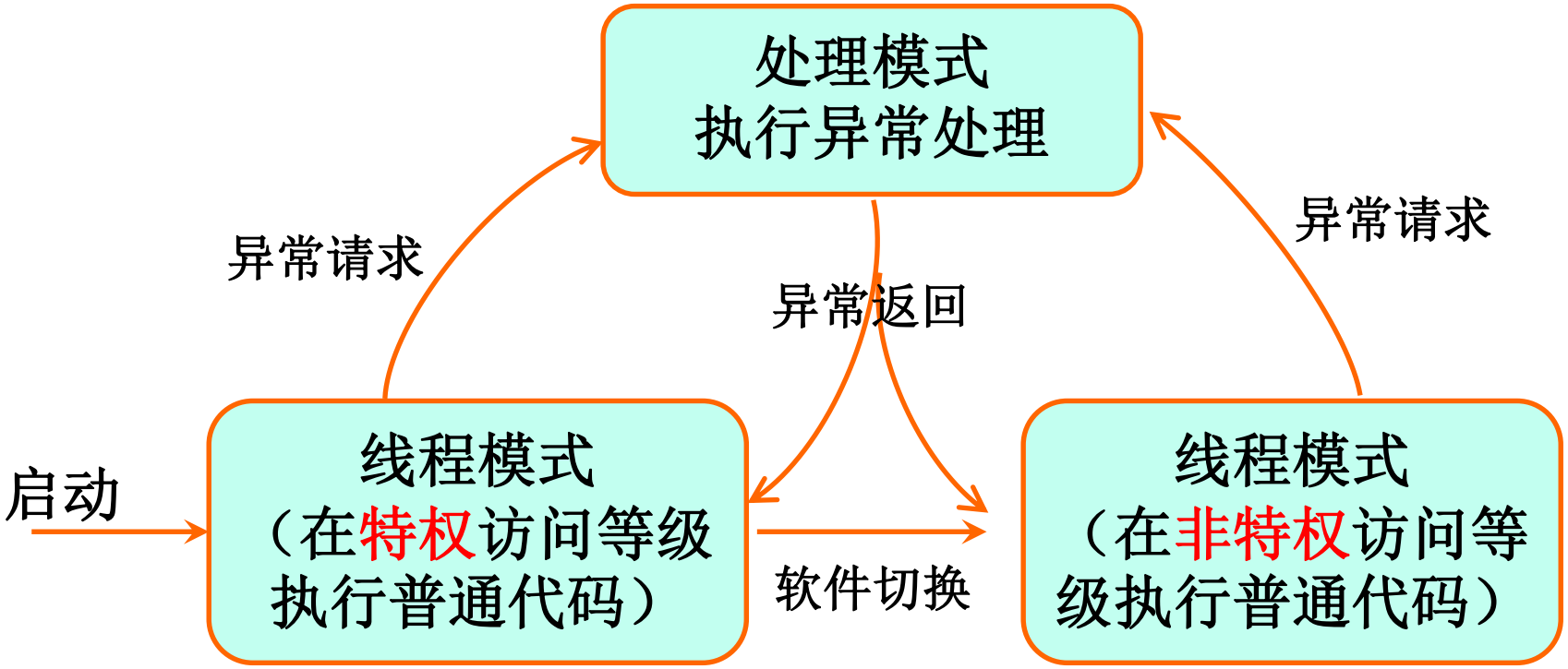
控制寄存器　CONTROL

用途：用于定义处理器的特权级别和选择使用哪个堆栈指针。



位	功能描述
nPRIV **CONTROL[0]**	定义线程模式的特权等级（注：处理模式永远都是特权级） 0（缺省）=处理器处于特权级的线程模式 1　　=处理器处于非特权级（用户级）的线程模式
SPSEL **CONTROL[1]**	堆栈指针选择 0=线程模式时选择主堆栈指针MSP（复位后的缺省值） 1=线程模式时选择进程堆栈指针PSP 在处理模式（Handler模式）下，只允许使用MSP。
FPCA **CONTROL[2]**	异常处理机制使用该位确定异常产生时，浮点单元的寄存器是否需要保存：该位为1时，当前使用浮点指令，需要保存浮点寄存器。

Cortex-M的两种工作模式



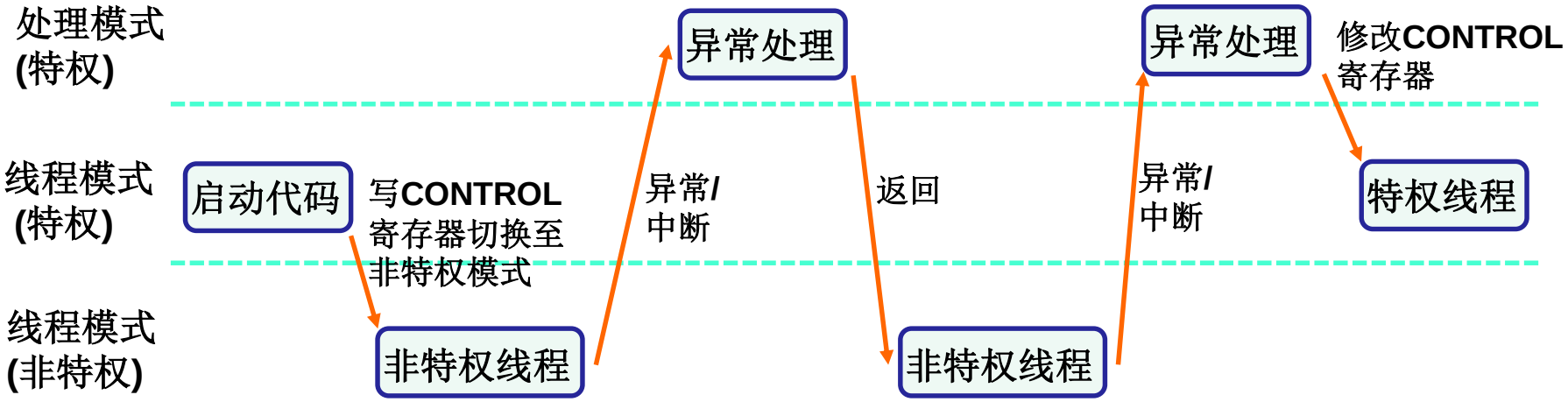
处理模式(Hander Mode)：执行中断服务程序等异常处理。该模式下，处理器总是处于特权访问等级。

线程模式(Thread Mode)：执行普通应用程序时，处理器可以处于特权访问等级，也可以处于非特权访问等级，由CONTRL寄存器控制。

Cortex-M的两种工作模式

控制寄存器的作用：为嵌入式操作系统OS的设计提供便利

运行在非特权等级的程序无法切换回特权访问模式，这样就提供了一个基本的安全模式。例如：嵌入式系统可能会具有运行在非特权访问等级的不受信任的应用，这些应用的访问权限就需要受到限制，以免不可靠的程序引起整个系统的崩溃。

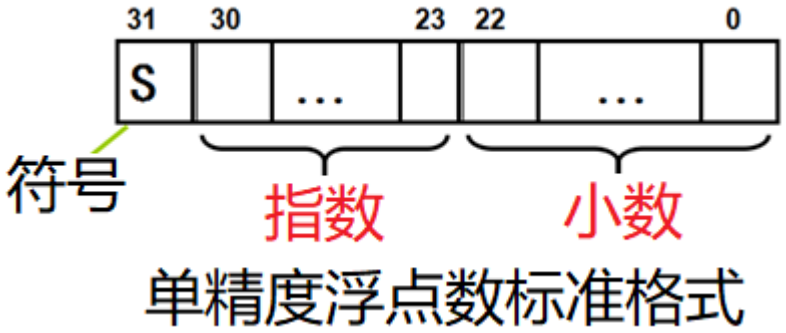


特权线程模式和非特权线程模式间的切换

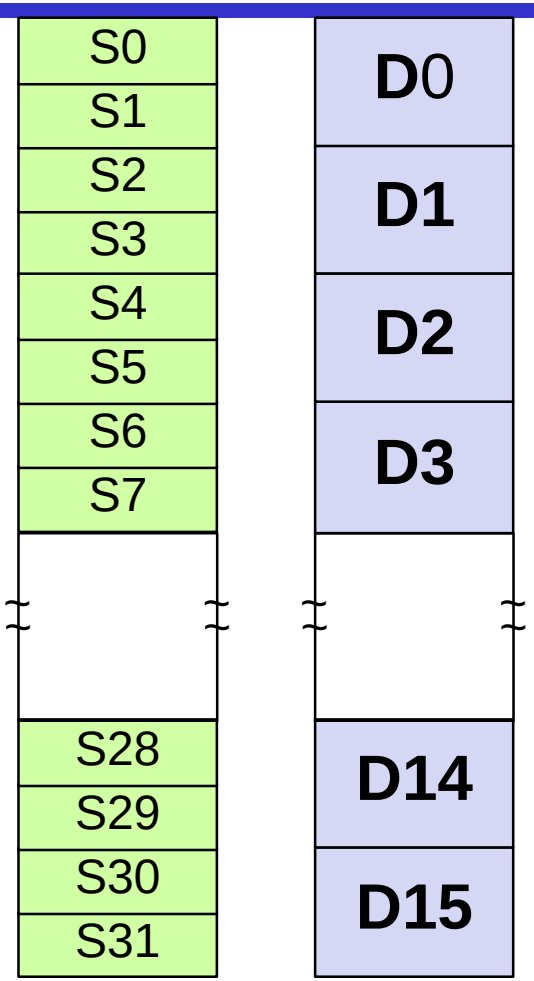
2.1 寄存器

Cortex-M4F 浮点寄存器

- FPU 提供32 个单精度寄存器
- 可以作为
  - 32 个 32-bit 寄存器
  - 16 个双字寄存器



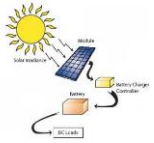
单精度浮点数标准格式



浮点状态寄存器 FPSCR

31	30	29	28	27	26	25	24	23:22	21:8	7	6:5	4	3	2	1	0
N	Z	C	V		AHP	DN	FZ	RMode	Reserved	IDC	Reserved	IXC	UFC	OFC	DZC	IOC

Reserved

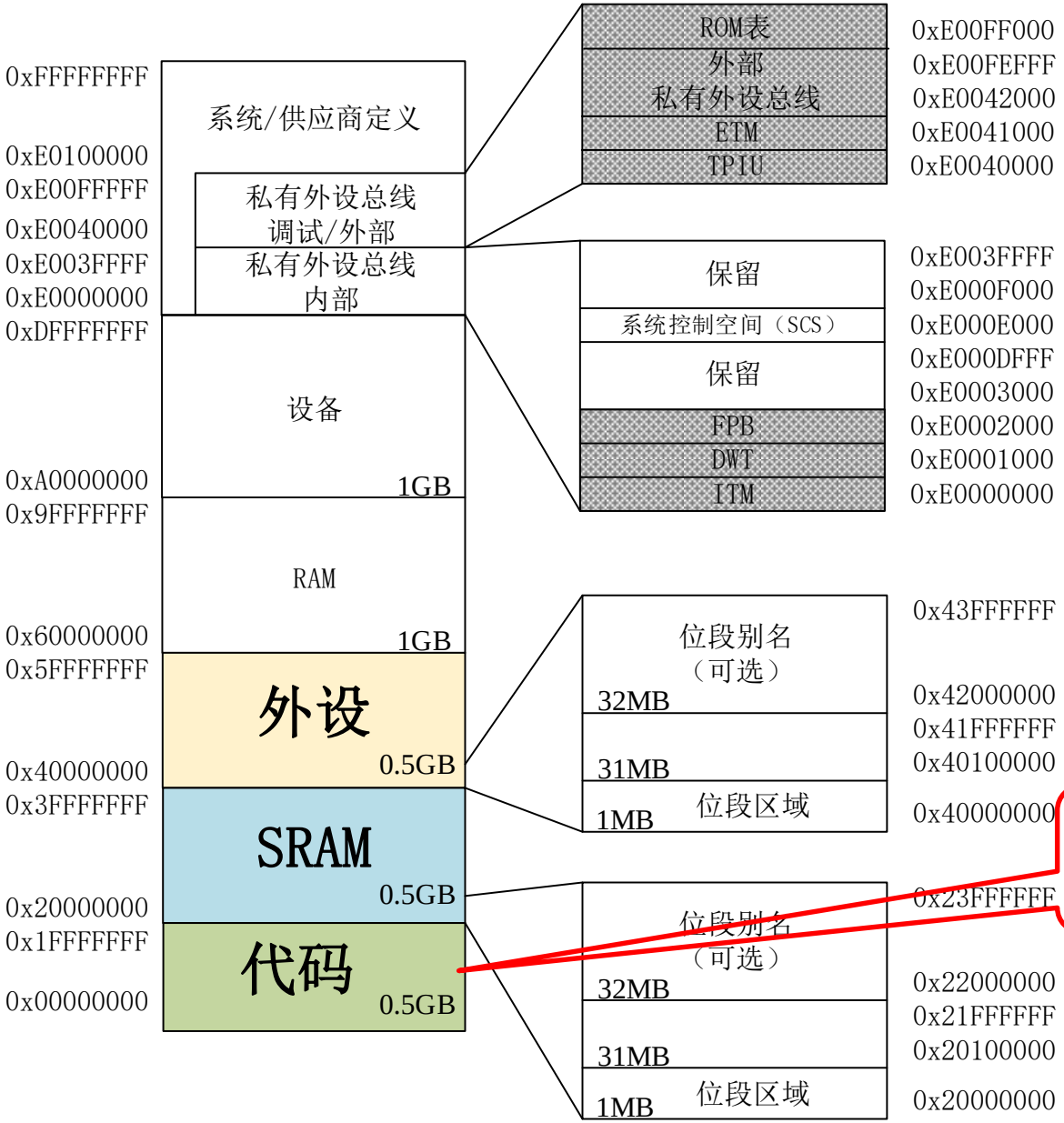


2.2 存贮器

特性：

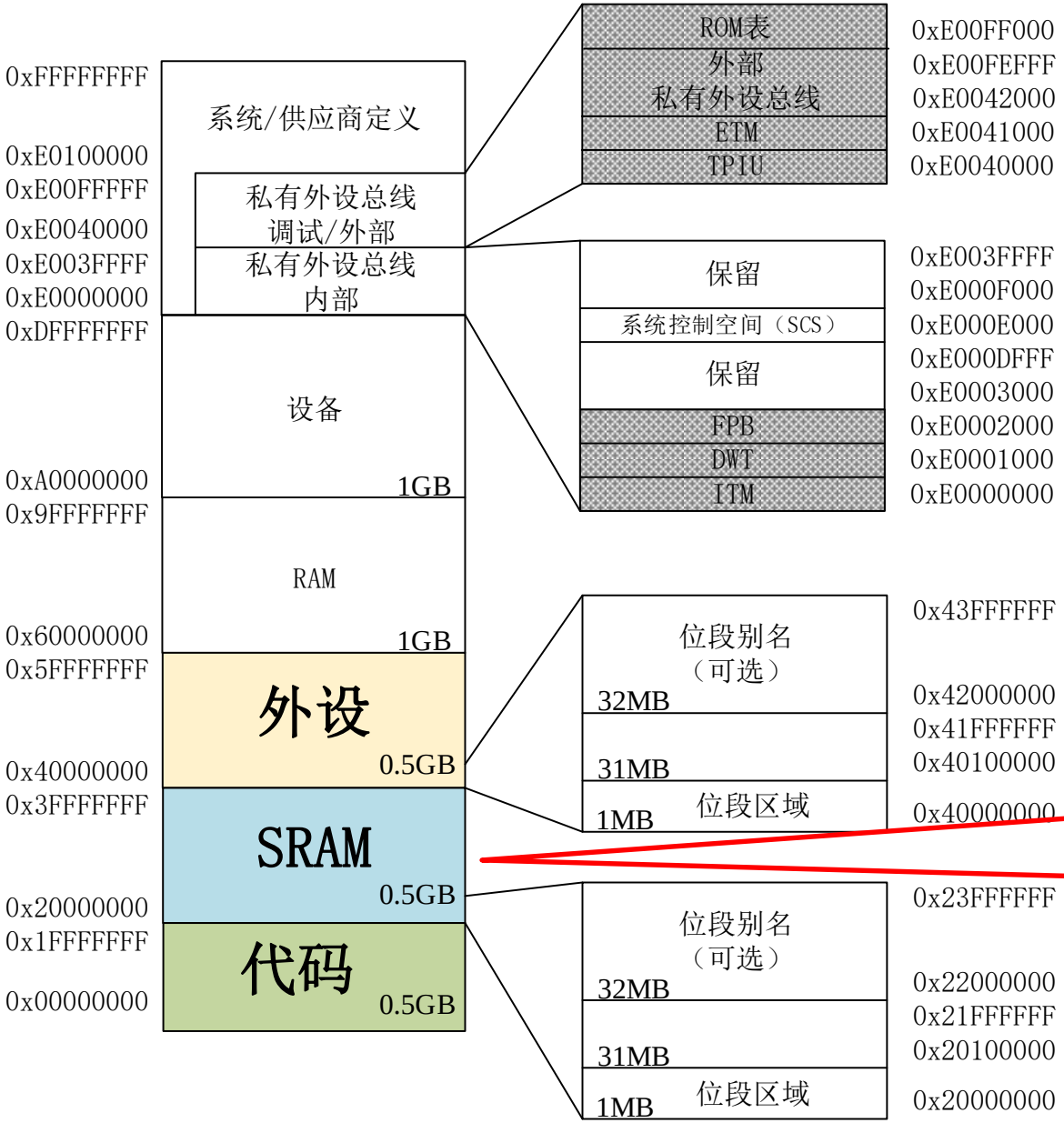
- 可访问多达4GB的存贮器空间
- 存贮器保护单元MPU，定义各存贮区域的访问权限，防止越界访问
- 根据典型用法，分为：
  - **程序代码访问区（如CODE区）**
  - **数据访问区　　（如RAM区）**
  - **外设**
  - **处理器的内部控制和调试部件**

存贮器映像（4GB）

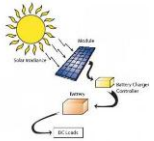


一般用于存放程序

存贮器映像（4GB）



一般用于存放数据，但也可以存放运行程序。

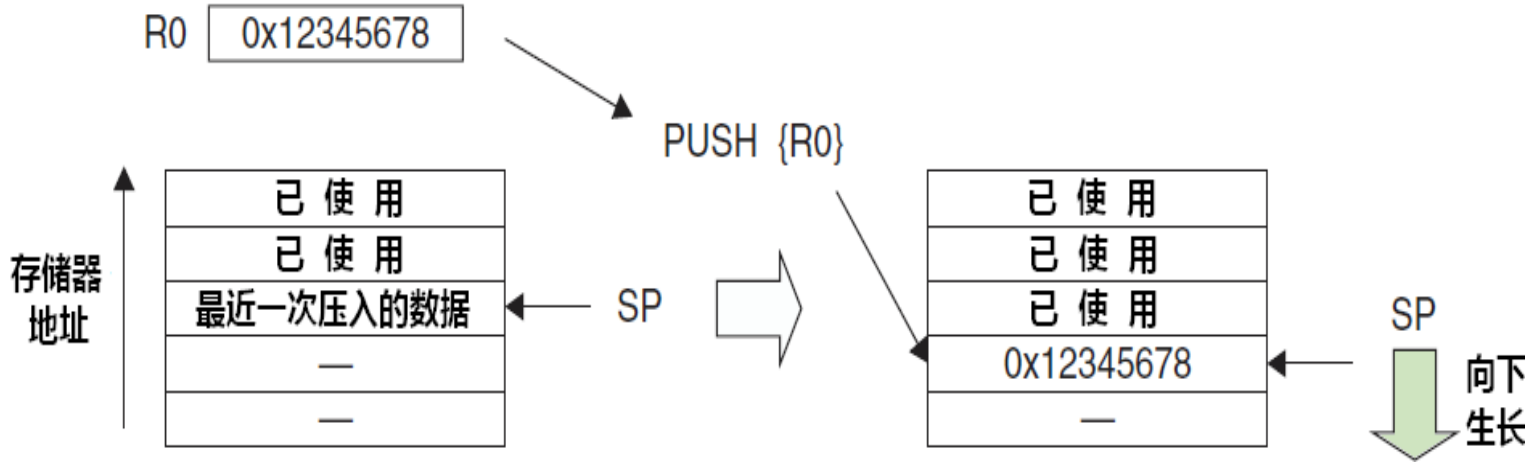


2.3 堆栈区

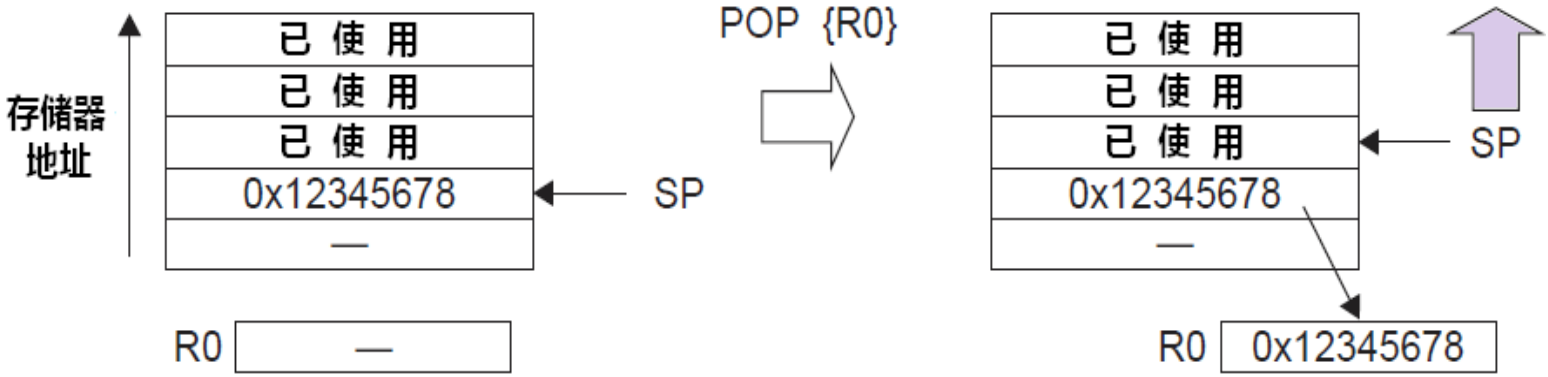
Cortex-M4使用的是“向下生长的满栈”模型。堆栈指针SP指向最后一个被压入堆栈的32位数值。在下一次压栈时，SP先自减4，再存入新的数值。

- 堆栈指针寄存器 （R13）
- PUSH 指令和 POP 指令
- 对32位操作，对齐到4字节边界，SP的最低两位总是为0
- 在物理上存在两个栈指针，主栈指针（MSP）和进程栈指针（PSP），PSP指针用于运行操作系统。

2.3 堆栈区



Cortex堆栈的PUSH操作



Cortex堆栈的POP操作

堆栈应用

主程序：

;;R4=X, R5=Y, R6=Z

BL　function1

;;返回主程序

;; R4=X, R5=Y, R6=Z

子程序：

function1

```
    PUSH   {R4}
    PUSH   {R5}
    PUSH   {R6}
    ……
    POP    {R6}
    POP    {R5}
    POP    {R4}
    BX     LR
```

;;返回

将多个寄存器压栈

执行任务，R4, R5,R6可能变化

BL 指令：把当前PC放到LR寄存器，再把function1的值赋给PC(R15)，相当于LCALL指令

BX 指令：
根据寄存器值跳转

堆栈应用

主程序：

;;R4=X, R5=Y, R6=Z

BL　function1

;;后续程序

子程序：

function1

PUSH　{R4-R6, LR}

……

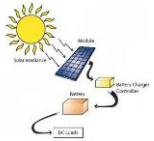
POP　{R4-R6, PC}

;;返回

将多个寄存器压栈

注意：PUSH　LR
　　POP　PC

汇编器将PUSH的寄存器按升序排序。即先PUSH序号大的寄存器，退出时POP序号小的寄存器。



（三）指令集

特点：

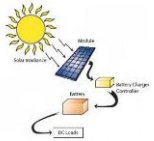
- 32位或16位的RISC指令集
- 指令丰富，功能强大

指令格式：

标号

操作码　操作数1，操作数2，…　　；注释

其中，标号是可选的，如果有，它**必须顶格写**。标号的作用是让汇编器来计算程序转移的地址。操作码是指令的助记符，它的前面**必须有至少一个空白符，**通常使用一至二个“Tab”键来产生。操作码后面往往跟随若干个操作数，而第1个操作数，通常都给出本指令的执行结果存储处。



（三）指令集

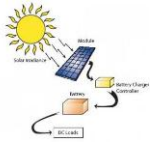
- 举例：立即数必须以“#”开头，如

```
MOV    R0,    #0x12  ; R0 ← 0x12
MOV    R1,    #'A'   ; R1 ← 字母A的ASCII码
```

注释均以”;”开头，它的有无不影响汇编器工作。

可以使用EQU指示字来定义常数，然后在代码中使用它们，例如：

```
NVIC_IRQ_SETEN0    EQU    0xE000E100    ; 常数定义必须顶格写
NVIC_IRQ0_ENABLE   EQU    0x1
        …
        LDR    R0,    =NVIC_IRQ_SETEN0    ;给R0赋值
        MOV    R1,    #NVIC_IRQ0_ENABLE   ; 把立即数传送到R1中
        STR    R1,    [R0]                ; *R0=R1，执行完此指
                                             令后IRQ #0被使能。
```



（三）指令集

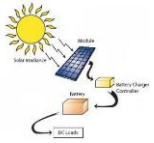
指令后缀：在ARM处理器中，指令可以带有后缀。

后缀名	含义
S	要求更新APSR中的相关标志。 例如： ADDS　R0,　R1　；根据加法的结果更新APSR中的标志
EQ **NE** **LT** **GT**	有条件地执行指令。EQ=Euqal, NE= Not Equal, LT= Less Than, GT= Greater Than，例如： BEQ <Label>　；仅当EQ满足时转移 除了这4种，还有若干个其它的条件。

Cortex中，条件后缀的使用有很大的限制：只有转移指令（B指令）才可随意使用。

APSR中的5个标志位

- **N: 负数标志(Negative)**
- **Z: 零结果标志(Zero)**
- **C: 进位/借位标志(Carry)**
- **V: 溢出标志(oVerflow)**
- **S: 饱和标志(Saturation)，它不做条件转移的依据**



（三）指令集

统一汇编语言（UAL）

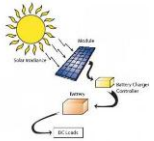
将传统Thumb指令移植到Thumb-2指令时存在不兼容的问题，UAL规范了汇编语言的格式。如传统Thumb下的ADD指令都默认更新APSR，因此在Thumb2都被更改为ADDS。

在 Thumb-2 指令集中，有些操作既可以由 16 位指令完成，也可以由 32 位指令完成。 例如，R0=R0+1 这样的操作，16 位的与 32 位的指令都提供了助记符为“ADD”的指令。在 UAL 下，你可以让汇编器决定用哪个，也可以手工指定是用 16 位的还是 32 位的。

```
ADDS.N   R0, #1        ;指定使用 16 位指令（N＝Narrow）
ADDS.W   R0, #1        ;指定使用 32 位指令（W=Wide）
ADDS     R0, #1        ;汇编器将为了节省空间而使用 16 位指令
```

注意：16 位指令与 32 位指令中的立即数最大允许长度不同！

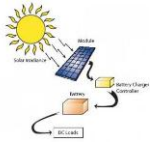
```
例：              MOV   R0, #direct
      16位指令    direct长度8位
      32位指令    direct长度16位
```



（三）指令集

指令类型

处理器内部传送数据
存贮器访问
算术运算
逻辑运算
移位和循环移位运算
转换（展开和反转运算）运算
位域处理指令
程序流控制
乘累加（MAC）指令
除法指令
……



（三）指令集

（1）处理器内部转移指令：

- 在两个寄存器间传送数据
- 在寄存器与特殊功能寄存器间传送数据
- 把一个立即数加载到寄存器
- 寄存器间传送数据的指令是MOV

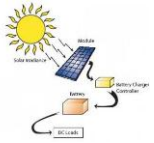
例：

```
MOV    R4，R0
MOVS   R4，#0x12          ;更新APSR标志
MOV    R4，# ’A’

MOVW   R4，#0x1234        ;将0x1234放到R4的低16位，高16位清空
MOVT    R4，#0x5678       ;把0x5678放到R4的高16位，低16位不影响
```

MOV的一个衍生指令是MVN，它把寄存器的内容取二进制补码后再传送。

```
MVN     R3，R4
```



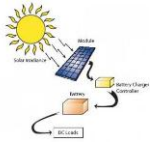
（三）指令集

（2）存贮器访问指令：

用于访问存储器的基础指令是“加载（Load）”和“存储（Store）”。加载指令LDR把存储器中的内容加载到寄存器中，存储指令STR则把寄存器的内容存储至存储器中。（存贮器访问指令的执行时间至少增加一个时钟周期）

各种数据大小的存贮器访问指令

数据类型	加载（读存贮器）	存贮（写存贮器）
8位无符号数	LDRB	STRB
8位有符号数	LDRSB	STRSB
16位无符号数	LDRH	STRH
16位有符号数	LDRSH	STRSH
32位	LDR	STR
双字	LDRD	STRD
多个32位	LDM	STM



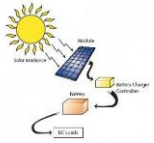
（三）指令集

（2）存贮器访问指令：立即数偏移（前序）

以一个寄存器作为存贮器的基准地址（指针），与一个立即数相加，其和作为访问存贮器的地址，这种指令称为“前序”寻址。

示例	功能描述
LDRB Rd, [Rn, #offset]	从地址Rn+offset处读取一个字节送到Rd
LDRH Rd, [Rn, #offset]	从地址Rn+offset处读取一个半字送到Rd
LDR Rd, [Rn, #offset]	从地址Rn+offset处读取一个字送到Rd
LDRD Rd1, Rd2, [Rn, #offset]	从地址Rn+offset处读取一个双字(64位整数)送到Rd1（低32位）和Rd2（高32位）中。 Offset长度：16位指令时5位；32位指令时12位
STRB Rd, [Rn, #offset]	**把Rd中的低字节存储到地址Rn+offset处**
STRH Rd, [Rn, #offset]	把Rd中的低半字存储到地址Rn+offset处
STR Rd, [Rn, #offset]	把Rd中的低字存储到地址Rn+offset处
STRD Rd1, Rd2, [Rn, #offset]!	把Rd1（低32位）和Rd2（高32位）表达的双字存储到地址Rn+offset处

注意：偏移量后面可加后缀(!) 表示指令执行完成后更新存放地址的寄存器



（三）指令集

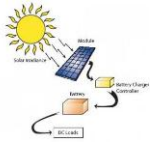
（2）存贮器访问指令：寄存器偏移（前序）

以一个寄存器作为存贮器的基准地址（指针），与另一个从索引寄存器值相加，其和作为访问存贮器的地址。从索引寄存器值在相加前，可左移0~3位。

示例	功能描述
LDRB Rd, [Rn，Rm{, LSL, #n}]	从存贮器位置Rn+(Rm<<n)处读取一个字节送到Rd
LDRH Rd, [Rn，Rm{, LSL, #n}]	从存贮器位置Rn+(Rm<<n)处读取一个半字送到Rd
LDR Rd, [Rn，Rm{, LSL, #n}]	从存贮器位置Rn+(Rm<<n)处读取一个字送到Rd
LDRSB Rd, [Rn，Rm{, LSL, #n}]	从从存贮器位置Rn+(Rm<<n)处读取一个字节并进行有符号展开。
STRB Rd, [Rn，Rm{, LSL, #n}]	往存贮器位置Rn+(Rm<<n)处存储一个字节
STRH Rd, [Rn，Rm{, LSL, #n}]	往存贮器位置Rn+(Rm<<n)处存储一个半字
STR Rd, [Rn，Rm{, LSL, #n}]	往存贮器位置Rn+(Rm<<n)处存储一个字

例：

```
LDRB    R0, [R5, R1, LSL #1];
STR     R0, [R1, R2, LSL #2];
```



（三）指令集

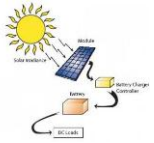
（2）存贮器访问指令：后序

该方式也有一个立即数偏移值，但是在存贮器访问期间不用偏移值，仅在数据结束后更新地址寄存器（指针）。指令无需用感叹号（！）。

示例	功能描述
LDRB Rd, [Rn]，#offset	读存贮器[Rn] 处的字节到Rd,然后更新Rn为Rn+offset
LDRH Rd, [Rn]，#offset	读存贮器[Rn] 处的半字到Rd,然后更新Rn为Rn+offset
LDR Rd, [Rn]，#offset	读存贮器[Rn] 处的字到Rd,然后更新Rn为Rn+offset
LDRSB Rd, [Rn]，#offset	读存贮器[Rn] 处的字节到Rd并进行有符号展开,然后更新Rn为Rn+offset。
STRB Rd, [Rn]，#offset	往存贮器位置[Rn]处存储一个字节,然后更新Rn为Rn+offset
STRH Rd, [Rn]，#offset	往存贮器位置[Rn]处存储一个半字,然后更新Rn为Rn+offset
STR Rd, [Rn]，#offset	往存贮器位置[Rn]处存储一个字,然后更新Rn为Rn+offset

例：

```
LDRB    R0, [R5], #4;
STR     R0, [R1], #20;
```



（三）指令集

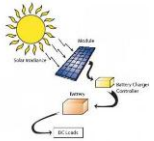
（2）存贮器访问指令：

例1：

```
LDRB    R4，[R1]          ;;从存贮器地址[R1]读一个字节存入R4
LDR     R0，[R1,#08]!
                  ;;在读取[R1+0x8]到R0后，R1被更新为R1+0x8
                  ;;感叹号(!) 表示指令执行完成后更新存放地址的
                    寄存器
STR     R0，[R1]          ;;将R0的内容写到[R1]地址
STRD    R0,R1,[R2,#08]!   ;;将R0，R1作为双字，写到
                          [R2+0x8]地址并更新R2
LDR     R0, [PC, #0x20]    ;;利用PC偏移加载字数据到R0

LDRNE   R0, [R5], #4;        ;;APSR的Z标志不为1执行该指令

LDR     R0, [R5, R1, LSL #2] ;
```



（三）指令集

（2）存贮器访问指令：

C语言对应的汇编

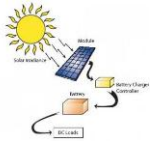
a=X[n];

设a对应R0， n对应R1， X为8位的数组，X[0]的地址存于R2

```
LDRB  R0, [R2,R1]
```

如X为32位的数组

```
LDRB  R0, [R2,R1, LSL#2]
```



（三）指令集

（2）存贮器访问指令：

C语言对应的汇编

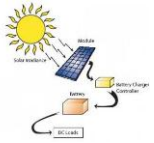
a=X[n++];

设a对应R0，X为32位的数组，X[n]的地址存于R2

```
LDR  R0, [R2], #4
```

a=X[++n];

```
LDR  R0, [R2, #4]!
```



（三）指令集

例2：对32位内存地址赋值

MOV Rn,#imm　只能赋值16位数，无法赋值32位数指令

有三种方法对32位地址赋值

（1）分成两条语句

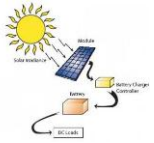
```
MOVW        R0, #0xE100    ;对低16位赋值
MOVT        R0, #0xE000    ;对高16位赋值
```

（2）采用ADR指令：　ADR　Rn, label

利用当前PC值，编译器计算偏移量得到需要地址。Label与PC的偏移量在-4095 ~ 4095之间。

```
    ADR   R0, address1    ; R0= 0x4000
        ……
 address1
0x4000: MOV R0, R1
```

ADR Rn, [PC] + #rel



（三）指令集

例2：对32位内存地址赋值

（3）LDR伪指令

```
NVIC_IRQ_SETEN      EQU     0xE000 E100
NVIC_IRQ0_ENABLE    EQU     0x1

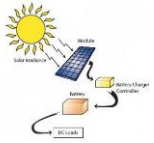
LDR     R0, = NVIC_IRQ_SETEN      ;;将0xE000 E1000放入R0
        ;;这里的LDR为伪指令，实际转换成
LDR     R0, [PC, #offset]

MOVS    R1, # NVIC_IRQ0_ENABLE
STR     R1, [R0]

Table:
DCD     0xE000 E100
```

伪指令

伪指令：定义表格数据



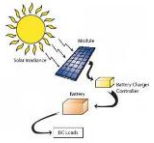
（三）指令集

多重存贮器访问指令：

可以读或写内存中的多个连续数据。

多重加载指令	功能描述
LDMIA　Rd!, {寄存器列表}	从Rd指定的存贮器地址读取多个字到寄存器列表，地址在每次读取后加4（IA）。
STMIA　Rd!, {寄存器列表}	依次将寄存器列表的值写入Rd指定的地址，地址在每次读取后加4（IA）。
LDMDB　Rd!, {寄存器列表}	从Rd指定的存贮器地址读取多个字到寄存器列表，地址在每次读取前减4（DB）。
STMDB　Rd!, {寄存器列表}	依次将寄存器列表的值写入Rd指定的地址，地址在每次读取前减4（DB）。

IA: Increase After　　DB:Decrease Before



（三）指令集

例3：读取地址0x20000000~0x2000000f的内容到R0~R3

```
LDR     R4，=0x20000000
LDMIA   R4，{R0-R3}
```

R4值不变

例4：

```
LDR     R8，=0x8000
STMIA   R8！{R0-R3}
```

R8=0x8010

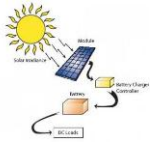
例5：

```
        STMDB   SP!,  {R0-R3, LR}
```

等效于 PUSH {R0-R3, LR}

```
        LDMIA   SP!,  {R0-R3, PC}
```

等效于 POP {R0-R3, PC}

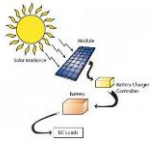


（三）指令集

（3）算术指令：

常见的算术四则运算指令

示例			功能描述
ADD	Rd, Rn, Rm	; Rd = Rn+Rm	常规加法 imm的范围是im8（16位指令）或im12（32位指令）
ADD	Rd, Rm	; Rd += Rm	
ADD	Rd, #imm	; Rd += imm	
ADC	Rd, Rn, Rm	; Rd = Rn+Rm+C	带进位的加法 imm的范围是im8（16位指令）或im12（32位指令）
ADC	Rd, Rm	; Rd += Rm+C	
ADC	Rd, #imm	; Rd += imm+C	
ADDW	Rd, #imm12	; Rd += imm12	带12位立即数的常规加法
SUB	Rd, Rn	; Rd -= Rn	常规减法
SUB	Rd, Rn, #imm3	; Rd = Rn-imm3	
SUB	Rd, #imm8	; Rd -= imm8	
SUB	Rd, Rn, Rm	; Rd = Rn-Rm	



（三）指令集

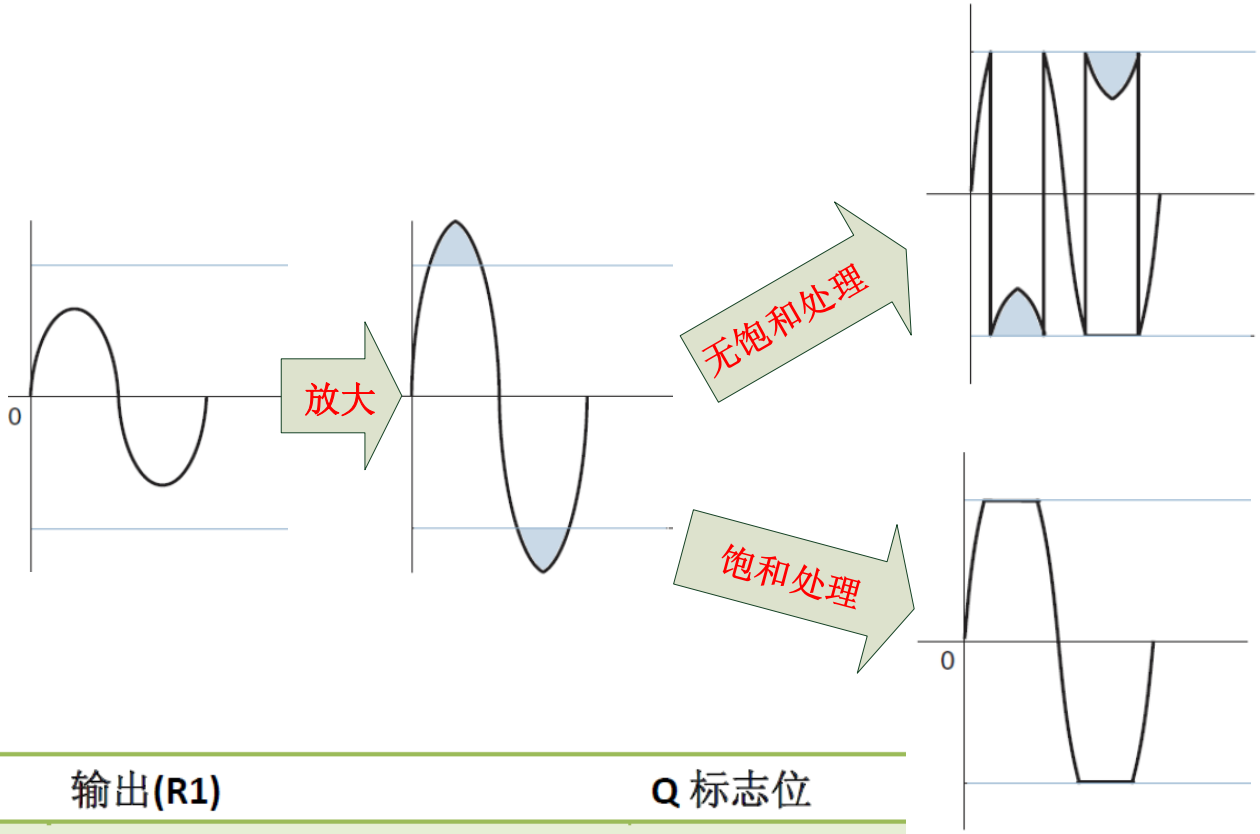
（3）算术指令：

SBC Rd, Rm ; Rd -= Rm+C **SBC.W Rd, Rn, #imm12 ; Rd = Rn-imm12-C** **SBC.W Rd, Rn, Rm ; Rd = Rn-Rm-C**	**带借位的减法**
RSB.W Rd, Rn, #imm12 ; Rd = imm12-Rn **RSB.W Rd, Rn, Rm ; Rd = Rm-Rn**	反向减法
MUL Rd, Rm ; Rd *= Rm **MUL.W Rd, Rn, Rm ; Rd = Rn*Rm**	常规乘法
MLA Rd, Rm, Rn, Ra ; Rd = Ra+Rm*Rn **MLS Rd, Rm, Rn, Ra ; Rd = Ra-Rm*Rn**	乘加与乘减
UDIV Rd, Rn, Rm ; Rd = Rn/Rm （无符号除法） **SDIV Rd, Rn, Rm ; Rd = Rn/Rm （带符号除法）**	硬件支持的除法

（三）指令集

饱和运算指令：

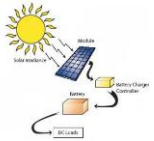
SSAT Rd, #n, Rm
QADD R7, R4, R2
QSUB R7, R4, R2



例：

SSAT R1, #12, R0

输入(R0)	输出(R1)	Q 标志位
0x2000(8192)	0x7FF(2047)	1
0x537(1335)	0x537(1335)	无变化
0x7FF(2047)	0x7FF(2047)	无变化
0	0	无变化
0xFFFFE000(-8192)	0xFFFFF800(-2048)	1
0xFFFFFB32(-1230)	0xFFFFFB32(-1230)	无变化



（三）指令集

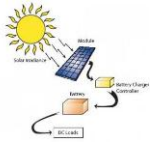
（3）算术指令：

例：

```
ADD     R2, R0, R1          ;;R2=R0+R1
ADDS    R2, R0, R1          ;;R2=R0+R1，APSR标志更新
ADC     R2, #0x100          ;;R2=R2+0x100+进位
SUB     R2,R1,#200          ;;R2=R1-200
MUL     R2, R1,R0           ;; R2=R1*R0
UDIV    R2, R1,R0           ;;无符号除法
```

乘加指令：

```
MLA    Rd,Rn,Rm,Ra          ;;Rd=Rn*Rm+Ra
MLS    Rd,Rn,Rm,Ra          ;; Rd=Ra-Rn*Rm
```



（三）指令集

（4）逻辑运算：

```
AND     R2, R0, R1              ;;R2=R0 &R1
ORR     R2, R1, #immed2         ;;R2=R1 | #immed2
BIC     R2,R1                   ;;R2=R2&(~R1)
EOR     R2,R0,R1                ;; R2=R0(XOR)R1
```

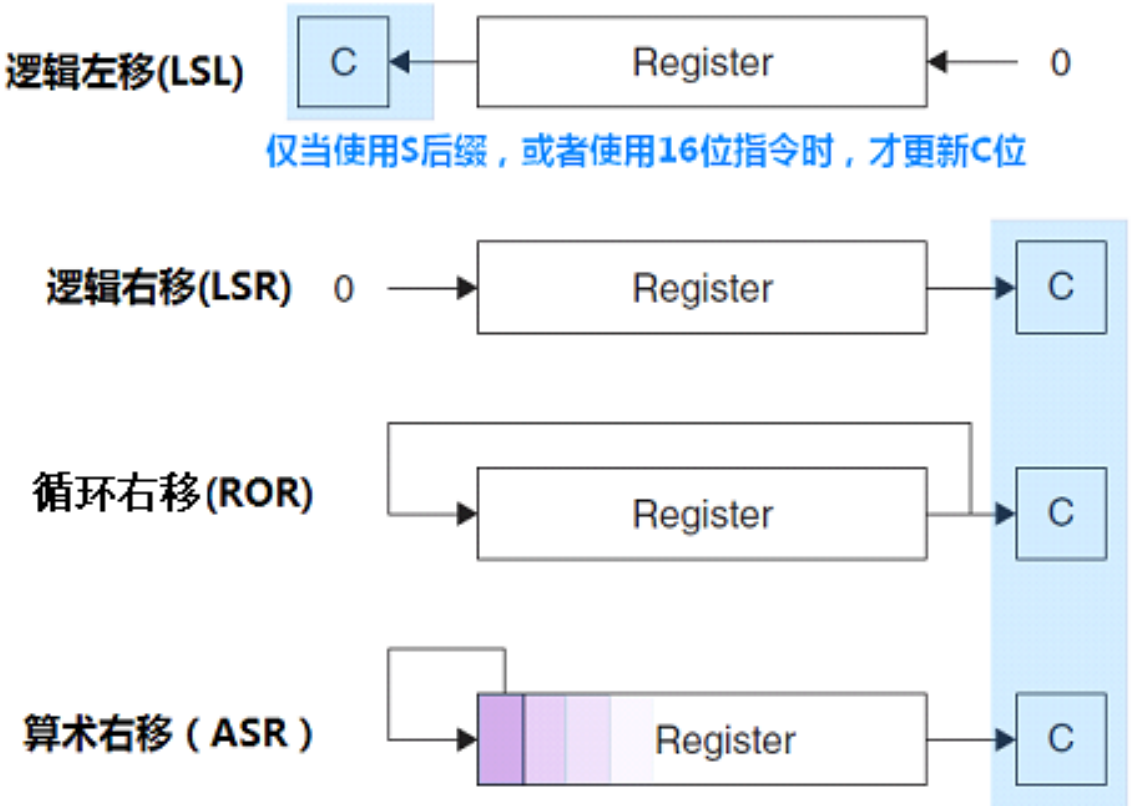
immed2的允许取值：

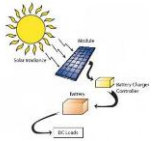
- 任意8位数左移得到的数；
- 0x00XY00XY 形式的数；
- 0xXY00XY00 形式的数；
- 0xXYXYXYXY 形式的数；

（三）指令集

（5）移位和循环移位：

```
LSL     R2, R1, R0          ;;R2=R1 <<R0        ;;逻辑左移
LSR     R2, R1, R0          ;;R2=R1 >>R0        ;;逻辑右移
ROR     R2, R1, R0          ;;R2=R1 <<R0        ;;循环右移
ASR     R2,R1,#3            ;;R2=R1 >>3         ;;算术右移
```





（三）指令集

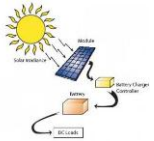
（6）位处理指令：

略

（7）数据转换指令：

用于数据格式转换，如大、小端数据转换

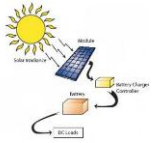
略



（三）指令集

（8）比较和测试：

```
CMP     R1,R0          ;;计算R1-R0，但不保存
CMP     R0, #0x60
CMN     R1,R0          ;;负比较，计算R1+R0
TST     R1,R2          ;;测试(按位与),APSR的N位和Z位更新
TEQ     R1,R2          ;;测试(异或),APSR的N位和Z位更新
```



（三）指令集

（9）程序流控制：

1、跳转

```
B       label                   ;;跳转到label，范围+-2KB
BX      R1                      ;;跳转到[R1]
```

2、函数调用

```
BL      label                   ;;跳转到label，并将返回地址保存LR
BLX     R1                      ;;跳转到[R1]，并将返回地址保存LR
```

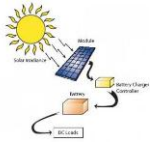
3、条件跳转

基于APSR的当前条件值执行（N、Z、C和V标志）

B<cond> <label>

<cond>为14个可能的条件后缀之一(EQ，NE，CS，CC，LT…)

例：
```
CMP   R0,#1
BEQ   label1
```



（三）指令集

（9）程序流控制：

4、比较和跳转

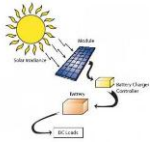
```
CBZ     R0, label          ;;R0=0跳转到label
CBNZ    R0, label          ;; R0不为零跳转到label
```

例：C语言

```
i=5;
while (i!=0){
        func1();
         i--;
}
```

对应汇编

```
        MOV   R0,  #5
loop1:
        CBZ     R0,loopexit
        BL      func1     ;;调用子程序
        SUBS  R0,#1
        B       loop1
loopexit:
```



（三）指令集

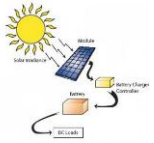
（9）程序流控制：

5、条件执行（IF-THEN block指令）最多4条件

```
IT  <cond>          ;;一个执行条件
ITE  <cond>         ;;两个执行条件
```

例：

```
CMP     R0, #1                  ;;
ITE     EQ                      ;;若Z置位(EQ)则执行下一条指令
                                ;;再往后一条在Z清零(NE)时执行
MOVEQ   R3,#2
MOVNE   R3,#1
```



（三）指令集

（9）程序流控制：

IT 指令优化C 代码的例子如以下伪代码所示：

```
if (R0==R1)
{
   R3 = R4 + R5;
   R3 = R3 / 2;
}
else
{
   R3 = R6 + R7;
   R3 = R3 * 2;
}
```

汇编：

```
CMP R0, R1                ; 比较R0 和R1
ITTEE EQ                  ; 如果 R0 == R1,
                          ;Then-Then-Else-Else
ADDEQ R3, R4, R5          ; 相等时加法
ASREQ R3, R3, #1          ; 相等时算术右移
ADDNE R3, R6, R7          ; 不等时加法
ADDNE R3, R3              ; 不等时乘2
```

（三）指令集

（10）C语言对应的汇编程序：

输入变量通过r0传递

C语言

```
int32_t  Madd( int a )
{
        int  b,c,d;
        b=24;
        c=36;
        d=a*b+c;
        return d;
}
```

汇编语言

```
PUSH    { lr }
MOVS    r1, r0
MOVS    r2,#0x18
MOVS    r3,#0x24
MLA     r0,r1,r2,r3
POP     {pc}
```

临时变量b,c,d存放在寄存器r1~r3

输出结果通过r0传递

（三）指令集

（10）C语言对应的汇编程序：

修改堆栈指针，存放临时变量

C语言

```
int  Madd( int a )
{
        int  b[20],c,d;
        b[0]=24;
        c=36;
        d=a*b[0]+c;
        return d;
}
```

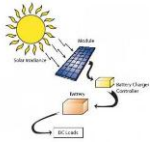
```
int32_t b[20],c,d;
        SUB             sp,sp,#0x50
        MOV             r1,r0
b[0]=24;
        MOVS            r3,#0x18
        STR             r3,[sp,#0x00]
c=36;
        MOVS            r2,#0x24
d=a*b[0]+c;
return d;
        LDR             r3,[sp,#0x00]
        MLA             r0,r1,r3,r2

        ADD             sp,sp,#0x50
        BX              lr
```

恢复堆栈指针

临时变量较多时，放在堆栈区

变量b[20]存放在堆栈区



作业：

1.讨论**Cortex M3/4** 的存贮器结构，哪些特点与传统冯.诺伊曼结构相似，哪些特点属于哈佛结构？

2. 两个**32**位数组**a[n]**和**b[n]**，存放在首地址为**0x2000 0000**和**0x2000 1000**的内存中，数组长度**N=32**。编写汇编程序计算两个数组的内积，即$\sum_{k=0}^{N-1} a[k] * b[k]$ ，结果存到**r10**（假设**a[n]**和**b[n]**的实际有效数据为**12**位，不考虑求和溢出情况）。要求尽量缩短程序执行时间，设时钟频率为**168MHz**，计算所编程序的大致执行时间。

提示：用乘加**MLA**指令